

EcoRide

Application de Covoiturage Écologique

DOSSIER DE PROJET

Évaluation en Cours de Formation (ECF)

Titre Professionnel — Développeur Web et Web Mobile (DWWM) — Niveau 5

Thomas Colin

Studi — Promotion 2026

Durée de développement : 3 semaines

Table des matières

Table des matières	2
Résumé	3
Remerciements	5
Introduction	6
1. Analyse et Conception du Projet	7
1.1. Présentation du projet et cahier des charges	7
1.2. Architecture du projet — Monorepo et pattern MVP	7
1.3. Choix technologiques	9
1.4. Modélisation de la base de données	9
2. Développement Front-End	12
2.1. Mise en place de l'environnement — React + Vite	12
2.2. Structure et organisation des composants React	13
2.3. Gestion de la navigation — React Router	14
2.4. Authentification côté client — JWT et Context API	15
2.5. Interface utilisateur — Tailwind CSS et Framer Motion	16
2.6. Gestion des formulaires et validation	18
2.7. Communication avec l'API	18
3. Développement Back-End	20
3.1. Architecture de l'API Express	20
3.2. Système d'authentification JWT	21
3.3. Middlewares d'authentification et de gestion des rôles	23
3.4. Contrôleurs et logique métier	24
4. Gestion des Données	26
4.1. Intégration Supabase — PostgreSQL	26
4.2. Intégration MongoDB — Journalisation des logs	27
4.3. Génération de données de test — Faker.js	28
5. Tests et Qualité du Code	30
5.1. Stratégie de tests	30
5.2. Tests unitaires avec Vitest	30
5.3. Résultats des tests	32
6. Déploiement et Infrastructure	33
6.1. Containerisation avec Docker	33
6.2. Déploiement du front-end sur Vercel	34
6.3. Déploiement du back-end sur Render	34
6.4. Gestion des variables d'environnement	35
Conclusion	37
Annexes	39
Annexe A — Schéma complet de la base de données	39
Annexe B — Points de terminaison de l'API (Routes)	39
Annexe C — Commandes utiles	40
Annexe D — Technologies utilisées (récapitulatif)	41

Résumé

EcoRide est une application web full-stack de covoiturage écologique, développée dans le cadre de l'Évaluation en Cours de Formation (ECF) du titre professionnel Développeur Web et Web Mobile (DWWM). L'objectif du projet est de proposer une plateforme permettant à des utilisateurs de publier ou de réserver des trajets partagés, en favorisant les véhicules électriques et les modes de transport éco-responsables.

Le projet a été réalisé en solo sur une durée de trois semaines, selon une architecture Monorepo regroupant un front-end React et une API back-end Express dans un même dépôt GitHub. Les principales fonctionnalités développées sont les suivantes :

- Authentification sécurisée par JSON Web Token (JWT) avec gestion des sessions côté client
- Système d'inscription et de connexion avec validation des données
- Gestion de trois rôles distincts : Visiteur, Utilisateur et Administrateur
- Publication, recherche et réservation de trajets avec système de filtres (écologique, prix, note)
- Système de crédits : chaque utilisateur reçoit 20 crédits à l'inscription et les utilise pour réserver des trajets
- Tableau de bord administrateur pour la gestion des utilisateurs et la consultation des statistiques
- Journalisation des actions d'administration via MongoDB

Sur le plan technique, le front-end repose sur React (avec Vite comme bundler), Tailwind CSS pour le style et Framer Motion pour les animations. Le back-end est une API REST développée avec Node.js et Express, connectée à deux bases de données : Supabase (PostgreSQL) pour les données relationnelles, et MongoDB pour les logs d'activité.

L'ensemble de l'application est conteneurisé avec Docker et déployé sur Vercel (front-end) et Render (back-end). Les tests unitaires et d'intégration sont réalisés avec Vitest.

Ce projet m'a permis de maîtriser l'ensemble du cycle de développement d'une application web moderne, de la conception de la base de données au déploiement en production, en passant par la sécurisation des routes API et la création d'interfaces utilisateur réactives.

Remerciements

Je tiens à remercier l'équipe pédagogique de Studi pour l'accompagnement et les ressources mis à disposition tout au long de cette formation. Leurs cours, exercices pratiques et retours réguliers m'ont permis de progresser rapidement et d'acquérir les compétences nécessaires à la réalisation de ce projet.

Je remercie également la communauté open source dont les bibliothèques, frameworks et outils constituent le socle technique d'EcoRide. Des projets tels que React, Express, Tailwind CSS ou Vitest sont le fruit d'un travail collaboratif considérable qui rend possible le développement d'applications modernes.

Enfin, je remercie les membres de ma famille et mes proches pour leur soutien moral pendant cette période de formation intensive.

Introduction

La mobilité durable est aujourd'hui un enjeu majeur de la transition écologique. Face à la hausse du coût des carburants, à la congestion des villes et aux impératifs environnementaux, le covoiturage s'impose comme une solution concrète et accessible. Dans ce contexte, le projet EcoRide a pour ambition de proposer une plateforme web intuitive, permettant à des particuliers de partager leurs trajets quotidiens ou longue distance, en privilégiant les véhicules propres.

Ce dossier de projet présente le travail réalisé dans le cadre de l'Évaluation en Cours de Formation (ECF) du titre professionnel Développeur Web et Web Mobile (DWWM, niveau 5 du Cadre National des Certifications). Il documente l'ensemble des étapes de développement : de la conception de l'architecture à la mise en ligne, en passant par le développement front-end et back-end, la gestion des données et les tests.

Le projet s'appuie sur une stack technologique moderne et représentative des standards de l'industrie web en 2026 : React pour l'interface, Node.js/Express pour l'API, PostgreSQL via Supabase et MongoDB pour les données, Docker pour la containerisation, et GitHub pour la gestion du code source. Chaque choix technologique sera justifié dans les sections correspondantes.

L'objectif de ce dossier est de démontrer la maîtrise des compétences requises par le référentiel DWWM, notamment le développement d'interfaces dynamiques, la création et l'interrogation de bases de données, la conception d'API REST sécurisées, et la mise en production d'une application complète.

1. Analyse et Conception du Projet

1.1. Présentation du projet et cahier des charges

EcoRide est une plateforme de covoiturage à destination du grand public. Son positionnement écologique la distingue des plateformes existantes : elle valorise les conducteurs utilisant des véhicules électriques ou hybrides et intègre des filtres de recherche basés sur les critères environnementaux.

Le cahier des charges de l'ECF définit les exigences suivantes :

- Permettre à un visiteur de consulter les trajets disponibles sans création de compte
- Permettre à un utilisateur inscrit de publier et de réserver des trajets
- Gérer un système de crédits : 20 crédits offerts à l'inscription, débités à chaque réservation
- Proposer un système d'avis après chaque trajet
- Fournir un espace d'administration pour la modération et les statistiques
- Sécuriser l'ensemble des routes sensibles par authentification JWT

Contrainte technique imposée

Le projet doit utiliser au minimum deux bases de données de nature différente (relationnelle et NoSQL) et être déployé sur des services cloud accessibles publiquement.

1.2. Architecture du projet — Monorepo et pattern MVP

J'ai fait le choix d'une architecture Monorepo, c'est-à-dire un unique dépôt GitHub qui contient à la fois le front-end, le back-end et les fichiers d'infrastructure (Docker). Ce choix présente plusieurs avantages :

- Centralisation du code et du versionning dans un seul endroit

- Facilité de partage de types et utilitaires communs
- Déploiement simplifié grâce à un seul pipeline CI/CD

L'architecture interne du back-end suit le pattern MVP (Model - View - Presenter), adapté au contexte d'une API REST :

- Model : représentation des entités de données et interactions avec la base de données
- View : dans le contexte d'une API, il s'agit des réponses JSON envoyées au client
- Presenter / Controller : logique métier, validation, orchestration des appels aux modèles

La structure des dossiers est la suivante :

ecoride/	
├─ docker-compose.yml	
├─ package.json	← Scripts racine (dev, test)
├─ .env	← Variables d'environnement (ignoré par Git)
├─ BackEnd/	
└─ src/	
└─ routes/	← Définition des routes Express
└─ controllers/	← Logique métier
└─ middlewares/	← Auth JWT, gestion des rôles
└─ models/	← Interactions BDD (Supabase, MongoDB)
└─ index.js	← Point d'entrée du serveur
└─ tests/	
└─ package.json	
└─ FrontEnd/	
└─ src/	
└─ components/	← Composants React réutilisables
└─ pages/	← Pages de l'application
└─ context/	← Contexte React (Auth, User)
└─ hooks/	← Hooks personnalisés
└─ main.jsx	← Point d'entrée React

└─ package.json

1.3. Choix technologiques

Chaque technologie a été choisie pour des raisons précises, en tenant compte à la fois des exigences du projet et des standards du marché.

Technologie	Rôle	Justification
React + Vite	Front-end SPA	Composants réutilisables, hot-reload rapide, écosystème riche
Tailwind CSS	Stylisation	Utilitaires CSS-in-HTML, cohérence visuelle, pas de conflits de classes
Framer Motion	Animations	API déclarative simple, transitions fluides sans CSS complexe
Node.js + Express	API Back-end	Performant, non-bloquant, idéal pour les API REST légères
JWT	Authentification	Sans état (stateless), sécurisé, standard de l'industrie
Supabase (PostgreSQL)	BDD relationnelle	PostgreSQL managé, SDK client, Row Level Security
MongoDB	Logs NoSQL	Flexibilité de schéma, performances d'écriture pour les logs
Docker	Infrastructure	Reproductibilité de l'environnement, isolation des services
Vitest	Tests	Compatible Vite, API Jest-like, rapide et moderne
Vercel + Render	Déploiement	Gratuits pour les projets solo, intégration GitHub native

1.4. Modélisation de la base de données

La base de données PostgreSQL (Supabase) centralise les données métier de l'application. Le schéma relationnel a été conçu pour répondre aux besoins fonctionnels tout en garantissant l'intégrité référentielle.

Les entités principales sont les suivantes :

- **users** : informations du compte (id, email, password_hash, role, credits, created_at)
- **rides** : trajets proposés (id, driver_id, origin, destination, departure_at, seats, price_per_seat, is_electric)
- **bookings** : réservations (id, ride_id, passenger_id, seats_reserved, status, created_at)
- **reviews** : avis (id, booking_id, author_id, target_id, rating, comment, created_at)
- **vehicles** : informations sur le véhicule d'un conducteur (id, user_id, brand, model, color, energy_type)

Les relations clés sont :

- Un utilisateur (user) peut avoir plusieurs trajets (rides) en tant que conducteur
- Un trajet peut avoir plusieurs réservations (bookings)
- Une réservation génère, après le trajet, la possibilité de laisser un avis (review)

-- Création de la table users	
CREATE TABLE users (	
id	UUID PRIMARY KEY DEFAULT gen_random_uuid(),
email	VARCHAR(255) UNIQUE NOT NULL,
password	TEXT NOT NULL,
username	VARCHAR(100) NOT NULL,
role	VARCHAR(20) DEFAULT 'user' CHECK (role IN ('user', 'admin', 'employee')),
credits	INTEGER DEFAULT 20,
created_at	TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);	

-- Création de la table rides	
CREATE TABLE rides (	
id	UUID PRIMARY KEY DEFAULT gen_random_uuid(),
driver_id	UUID REFERENCES users(id) ON DELETE CASCADE,
origin	VARCHAR(255) NOT NULL,
destination	VARCHAR(255) NOT NULL,
departure_at	TIMESTAMP WITH TIME ZONE NOT NULL,
seats_available	INTEGER NOT NULL CHECK (seats_available >= 0),
price_per_seat	DECIMAL(10,2) NOT NULL,
is_electric	BOOLEAN DEFAULT FALSE,
status	VARCHAR(20) DEFAULT 'active',
created_at	TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);	

2. Développement Front-End

2.1. Mise en place de l'environnement — React + Vite

Le front-end de l'application est une Single Page Application (SPA) initialisée avec Vite, un bundler de nouvelle génération qui offre un démarrage à froid quasi-instantané et un hot-module replacement (HMR) très rapide grâce à son utilisation des modules ES natifs du navigateur.

L'initialisation du projet s'effectue avec la commande suivante :

<code>npm create vite@latest FrontEnd -- --template react</code>
<code>cd FrontEnd</code>
<code>npm install</code>
<code>npm install tailwindcss framer-motion react-router-dom axios</code>
<code>npx tailwindcss init -p</code>

La configuration de Tailwind CSS dans vite.config.js permet d'activer le purge automatique des classes CSS inutilisées en production, réduisant significativement la taille du bundle final :

<code>// tailwind.config.js</code>
<code>export default {</code>
<code> content: ['./index.html', './src/**/*..{js,jsx}'],</code>
<code> theme: {</code>
<code> extend: {</code>
<code> colors: {</code>
<code> primary: '#2D7A2D',</code>
<code> 'primary-light': '#EAF5EA',</code>
<code> },</code>
<code> },</code>
<code> },</code>
<code> plugins: [],</code>
<code>};</code>

2.2. Structure et organisation des composants React

L'organisation du code front-end suit le principe de séparation des responsabilités. Les composants sont divisés en deux catégories : les composants « smart » (avec logique) et les composants « dumb » (purement présentationnels).

Les composants réutilisables (Button, Input, Card, Modal) sont regroupés dans le dossier /components, tandis que les pages complètes (HomePage, SearchPage, AccountPage) sont dans /pages. Cette séparation facilite la maintenance et les tests.

```
// src/components/ui/Button.jsx

const Button = ({ children, variant = 'primary', onClick, disabled, className = ''
}) => {

  const base = 'px-6 py-2 rounded-full font-medium transition-all duration-200';
  const variants = {
    primary: 'bg-primary text-white hover:bg-green-700',
    outline: 'border-2 border-primary text-primary hover:bg-primary-light',
    danger: 'bg-red-500 text-white hover:bg-red-600',
  };

  return (
    <button
      className={` ${base} ${variants[variant]} ${className}`}
      onClick={onClick}
      disabled={disabled}
    >
      {children}
    </button>
  );
};

export default Button;
```

2.3. Gestion de la navigation — React Router

La navigation entre les pages est gérée par React Router v6. Les routes sont définies de manière centralisée dans le fichier App.jsx, avec une distinction entre les routes publiques (accessibles à tous) et les routes protégées (nécessitant une authentification).

```
// src/App.jsx

import { BrowserRouter, Routes, Route } from 'react-router-dom';
import ProtectedRoute from '../components/ProtectedRoute';

function App() {
  return (
    <BrowserRouter>
      <Routes>
        {/* Routes publiques */}
        <Route path="/" element={<HomePage />} />
        <Route path="/search" element={<SearchPage />} />
        <Route path="/auth" element={<AuthPage />} />

        {/* Routes protégées (utilisateur connecté) */}
        <Route element={<ProtectedRoute />>
          <Route path="/account" element={<AccountPage />} />
          <Route path="/publish" element={<PublishRidePage />} />
        </Route>

        {/* Routes admin */}
        <Route element={<ProtectedRoute requiredRole='admin' />>
          <Route path="/admin" element={<AdminDashboard />} />
        </Route>
      </Routes>
    </BrowserRouter>
  );
}
```

Le composant `ProtectedRoute` intercepte la navigation et redirige vers la page d'authentification si l'utilisateur n'est pas connecté, ou vers une page d'erreur 403 s'il ne dispose pas du rôle requis.

2.4. Authentification côté client — JWT et Context API

La gestion de l'état d'authentification repose sur le Context API de React, qui permet de partager le token JWT et les informations de l'utilisateur connecté dans l'ensemble de l'arborescence de composants sans passer par des props.

```
// src/context/AuthContext.jsx

import { createContext, useContext, useState, useEffect } from 'react';

const AuthContext = createContext(null);

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
  const [token, setToken] = useState(localStorage.getItem('token'));

  // Décodage du payload JWT pour récupérer les infos utilisateur
  useEffect(() => {
    if (token) {
      try {
        const payload = JSON.parse(atob(token.split('.')[1]));
        if (payload.exp * 1000 < Date.now()) {
          logout(); // Token expiré
        } else {
          setUser(payload);
        }
      } catch {
        logout();
      }
    }
  });
}
```

<code>}, [token]);</code>
<code>const login = (newToken) => {</code>
<code> localStorage.setItem('token', newToken);</code>
<code> setToken(newToken);</code>
<code>};</code>
<code>const logout = () => {</code>
<code> localStorage.removeItem('token');</code>
<code> setToken(null);</code>
<code> setUser(null);</code>
<code>};</code>
<code>return (</code>
<code> <AuthContext.Provider value={{ user, token, login, logout }}></code>
<code> {children}</code>
<code></AuthContext.Provider></code>
<code>);</code>
<code>};</code>
<code>export const useAuth = () => useContext(AuthContext);</code>

Le token JWT est stocké dans le localStorage du navigateur et envoyé dans le header Authorization de chaque requête API nécessitant une authentification. J'ai conscience que le localStorage est vulnérable aux attaques XSS ; une amélioration future consisterait à utiliser des cookies HTTPOnly pour un stockage plus sécurisé.

2.5. Interface utilisateur — Tailwind CSS et Framer Motion

Tailwind CSS est utilisé pour l'intégralité des styles de l'application. Son approche « utility-first » permet de construire des interfaces cohérentes rapidement, sans avoir à

maintenir des feuilles de styles volumineuses. La palette de couleurs personnalisée (vert primaire #2D7A2D) est définie dans `tailwind.config.js` et réutilisée dans toute l'application.

Framer Motion apporte une couche d'animation déclarative. Les transitions de page et l'apparition progressive des cartes de trajets sont gérées avec ce library :

<code>// src/components/RideCard.jsx</code>
<code>import { motion } from 'framer-motion';</code>
<code></code>
<code>const RideCard = ({ ride, index }) => (</code>
<code> <motion.div</code>
<code> initial={{ opacity: 0, y: 20 }}</code>
<code> animate={{ opacity: 1, y: 0 }}</code>
<code> transition={{ duration: 0.3, delay: index * 0.1 }}</code>
<code> className='bg-white rounded-xl shadow-md p-6 hover:shadow-lg transition-shadow'</code>
<code>></code>
<code> <div className='flex justify-between items-center'></code>
<code> <div></code>
<code> <p className='text-xl font-bold</code>
<code>text-gray-800'>{ride.departure_at.slice(11,16)} →</p></code>
<code> <p className='text-sm text-gray-500'>{ride.seats_available} place(s)</code>
<code>disponible(s)</p></code>
<code></div></code>
<code> <div className='text-right'></code>
<code> <p className='text-2xl font-bold text-gray-900'>{ride.price_per_seat} €</p></code>
<code>{ride.is_electric && (</code>
<code> <span className='text-xs bg-green-100 text-green-700 px-2 py-1</code>
<code>rounded-full'></code>
<code> ♻ Écologique</code>
<code></code>
<code>) }</code>
<code></div></code>
<code></div></code>
<code></motion.div></code>
<code>);</code>

2.6. Gestion des formulaires et validation

Les formulaires (inscription, connexion, publication de trajet) implémentent une validation côté client avant l'envoi de la requête au serveur. Cette approche offre un retour immédiat à l'utilisateur et réduit la charge sur l'API.

```
// Exemple de validation du formulaire d'inscription
const validateRegister = (data) => {
  const errors = {};
  if (!data.email || !/\S+@\S+\.\S+/.test(data.email)) {
    errors.email = 'Adresse email invalide.';
  }
  if (!data.password || data.password.length < 8) {
    errors.password = 'Le mot de passe doit contenir au moins 8 caractères.';
  }
  if (data.password !== data.confirm) {
    errors.confirm = 'Les mots de passe ne correspondent pas.';
  }
  if (!data.username || data.username.trim().length < 2) {
    errors.username = 'Le pseudonyme est requis (minimum 2 caractères).';
  }
  return errors;
};
```

2.7. Communication avec l'API

Toutes les requêtes HTTP vers l'API back-end sont centralisées dans un fichier api.js utilisant Axios. Un intercepteur Axios ajoute automatiquement le token JWT dans le header de chaque requête, évitant de le répéter dans chaque appel.

```
// src/api/api.js
import axios from 'axios';

const api = axios.create({
```

```
    baseURL: import.meta.env.VITE_API_URL || 'http://localhost:3001',
    timeout: 10000,
  });

  // Intercepteur de requête : injection du token JWT
  api.interceptors.request.use((config) => {
    const token = localStorage.getItem('token');
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  });

  // Intercepteur de réponse : gestion globale des erreurs 401
  api.interceptors.response.use(
    (response) => response,
    (error) => {
      if (error.response?.status === 401) {
        localStorage.removeItem('token');
        window.location.href = '/auth';
      }
      return Promise.reject(error);
    }
  );
};

export default api;
```

3. Développement Back-End

3.1. Architecture de l'API Express

L'API back-end est développée avec Node.js et le framework Express. Elle suit une architecture RESTful : les ressources sont identifiées par des URL sémantiques, et les opérations CRUD sont mappées sur les verbes HTTP (GET, POST, PUT, DELETE).

Le point d'entrée du serveur (index.js) configure les middlewares globaux et monte les routeurs :

// BackEnd/src/index.js
const express = require('express');
const cors = require('cors');
const helmet = require('helmet');
const authRoutes = require('./routes/auth.routes');
const rideRoutes = require('./routes/rides.routes');
const userRoutes = require('./routes/users.routes');
const adminRoutes = require('./routes/admin.routes');
const app = express();
// Middlewares globaux
app.use(helmet()); // En-têtes de sécurité HTTP
app.use(cors({ origin: process.env.FRONTEND_URL }));
app.use(express.json()); // Parsing du body JSON
// Montage des routes
app.use('/api/auth', authRoutes);
app.use('/api/rides', rideRoutes);
app.use('/api/users', userRoutes);
app.use('/api/admin', adminRoutes);

```
// Middleware de gestion des erreurs globales
app.use((err, req, res, next) => {
  console.error(err.stack);

  res.status(err.status || 500).json({ error: err.message || 'Erreur interne du
  serveur' });
});

const PORT = process.env.PORT || 3001;
app.listen(PORT, () => console.log(`Serveur démarré sur le port ${PORT}`));
```

Bonne pratique : helmet.js

La bibliothèque Helmet configure automatiquement plusieurs en-têtes HTTP de sécurité (Content-Security-Policy, X-Frame-Options, X-XSS-Protection, etc.), protégeant l'API contre les attaques courantes sans configuration manuelle.

3.2. Système d'authentification JWT

L'authentification repose sur les JSON Web Tokens (JWT). À la connexion, le serveur génère un token signé contenant le payload de l'utilisateur (id, email, role). Ce token est renvoyé au client, qui le stocke et l'envoie dans le header de chaque requête protégée.

```
// Backend/src/controllers/auth.controller.js
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');
const { supabase } = require('../models/supabase');

const login = async (req, res) => {
  const { email, password } = req.body;

  if (!email || !password) {
    return res.status(400).json({ error: 'Email et mot de passe requis.' });
  }
}
```

// 1. Récupérer l'utilisateur depuis Supabase
const { data: user, error } = await supabase
.from('users')
.select('*')
.eq('email', email.toLowerCase())
.single();
if (error !user) {
return res.status(401).json({ error: 'Identifiants incorrects.' });
}
// 2. Vérifier le mot de passe haché
const valid = await bcrypt.compare(password, user.password);
if (!valid) {
return res.status(401).json({ error: 'Identifiants incorrects.' });
}
// 3. Générer le token JWT
const token = jwt.sign(
{ id: user.id, email: user.email, role: user.role, username: user.username },
process.env.JWT_SECRET,
{ expiresIn: '24h' })
);
res.json({ token, user: { id: user.id, email: user.email, role: user.role } });
};
module.exports = { login, register };

Le mot de passe n'est jamais stocké en clair. Lors de l'inscription, il est haché avec bcrypt (facteur de coût 12) avant d'être sauvegardé en base. Lors de la connexion, bcrypt.compare()

permet de vérifier le mot de passe saisi par rapport au hash stocké, sans jamais décrypter ce dernier.

3.3. Middlewares d'authentification et de gestion des rôles

Les middlewares sont des fonctions intermédiaires exécutées avant les contrôleurs. Deux middlewares clés sécurisent les routes de l'application :

```
// BackEnd/src/middlewares/auth.middleware.js

const jwt = require('jsonwebtoken');

// Vérifie la présence et la validité du token JWT
const authenticate = (req, res, next) => {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1]; // Bearer <token>

  if (!token) return res.status(401).json({ error: 'Token manquant.' });

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded; // Injecte les infos user dans la requête
    next();
  } catch (err) {
    return res.status(403).json({ error: 'Token invalide ou expiré.' });
  }
};

// Vérifie que l'utilisateur a le rôle requis
const authorize = (...roles) => (req, res, next) => {
  if (!roles.includes(req.user?.role)) {
    return res.status(403).json({ error: 'Accès non autorisé.' });
  }

  next();
};
```

```
};  
  
module.exports = { authenticate, authorize };
```

Ces middlewares sont ensuite composés dans les fichiers de routes. Par exemple, la route de création d'un trajet exige que l'utilisateur soit authentifié, et la route d'administration exige en plus le rôle 'admin' :

```
// BackEnd/src/routes/rides.routes.js  
  
const router = require('express').Router();  
  
const { authenticate, authorize } = require('../middlewares/auth.middleware');  
const ctrl = require('../controllers/rides.controller');  
  
  
router.get('/', ctrl.getRides); // Public  
router.get('/:id', ctrl.getRideById); // Public  
router.post('/', authenticate, ctrl.createRide); // Utilisateur connecté  
router.delete('/:id', authenticate, authorize('admin'), ctrl.deleteRide); // Admin seul  
  
  
module.exports = router;
```

3.4. Contrôleurs et logique métier

Chaque contrôleur gère une action spécifique sur une ressource. L'exemple ci-dessous illustre la création d'un trajet, incluant la validation des données et la sauvegarde en base :

```
// BackEnd/src/controllers/rides.controller.js  
  
const { supabase } = require('../models/supabase');  
  
  
const createRide = async (req, res) => {  
  const { origin, destination, departure_at, seats, price_per_seat, is_electric } = req.body;  
  
  const driver_id = req.user.id;
```

// Validation des données
if (!origin !destination !departure_at !seats !price_per_seat) {
return res.status(400).json({ error: 'Tous les champs obligatoires sont requis.' });
}
if (new Date(departure_at) <= new Date()) {
return res.status(400).json({ error: 'La date de départ doit être dans le futur.' });
}
const { data, error } = await supabase.from('rides').insert([
driver_id, origin, destination, departure_at,
seats_available: seats,
price_per_seat,
is_electric: is_electric false,
]).select();
if (error) return res.status(500).json({ error: error.message });
res.status(201).json(data[0]);
};

4. Gestion des Données

4.1. Intégration Supabase — PostgreSQL

Supabase est un Backend-as-a-Service (BaaS) open source qui fournit une base de données PostgreSQL managée, un système d'authentification, et une API REST auto-générée. Dans ce projet, j'utilise Supabase uniquement comme hébergeur PostgreSQL, tout en gérant l'authentification moi-même via JWT.

La connexion à Supabase s'effectue via le SDK officiel `@supabase/supabase-js` :

```
// BackEnd/src/models/supabase.js
const { createClient } = require('@supabase/supabase-js');

const supabase = createClient(
  process.env.SUPABASE_URL,
  process.env.SUPABASE_KEY
);

module.exports = { supabase };
```

Les requêtes sont construites de façon fluide grâce à l'API du SDK. L'exemple suivant illustre la recherche de trajets avec filtres :

```
// Recherche de trajets avec filtres
const getRides = async (req, res) => {
  const { origin, destination, date, is_electric } = req.query;

  let query = supabase
    .from('rides')
    .select(`*, driver:users(username, avatar_url)`)
    .eq('status', 'active')
    .gt('seats_available', 0);
```

if (origin) query = query.ilike('origin', `>\${origin}%`);
if (destination) query = query.ilike('destination', `>\${destination}%`);
if (date) query = query.gte('departure_at', date).lt('departure_at', date + 'T23:59:59');
if (is_electric === 'true') query = query.eq('is_electric', true);
const { data, error } = await query.order('departure_at', { ascending: true });
if (error) return res.status(500).json({ error: error.message });
res.json(data);
};

4.2. Intégration MongoDB — Journalisation des logs

MongoDB est utilisé en complément de PostgreSQL pour stocker les logs d'activité de l'espace d'administration. Ce choix est justifié par la nature flexible des logs : chaque action peut avoir des attributs différents, et la vitesse d'écriture est prioritaire sur la cohérence transactionnelle.

// BackEnd/src/models/mongo.js
const { MongoClient } = require('mongodb');
let db;
const connectMongo = async () => {
const client = new MongoClient(process.env.MONGO_URI);
await client.connect();
db = client.db('ecoride_logs');
console.log('MongoDB connecté');
};
const writeLog = async (action, userId, details = {}) => {
if (!db) return;

await db.collection('activity_logs').insertOne({
action, // ex : 'USER_SUSPENDED', 'RIDE_DELETED'
userId, // ID de l'admin ayant effectué l'action
details, // Données contextuelles (ID cible, raison...)
timestamp: new Date(),
});
};
module.exports = { connectMongo, writeLog };

Les logs sont consultables depuis le tableau de bord administrateur. Chaque action sensible (suspension d'utilisateur, suppression de trajet, modification de crédits) déclenche l'écriture d'un log via la fonction `writeLog()`. Cette traçabilité est essentielle pour l'audit et la modération de la plateforme.

4.3. Génération de données de test — Faker.js

Pour tester l'application avec des données réalistes, j'ai utilisé la bibliothèque `Faker.js` qui génère des données aléatoires mais cohérentes (noms, emails, adresses, dates, etc.). Des scripts de seeding permettent de peupler la base de données en une seule commande.

// BackEnd/src/seeds/seedRides.js
const { faker } = require('@faker-js/faker/locale/fr');
const { supabase } = require('../models/supabase');
const CITIES = ['Paris', 'Lyon', 'Marseille', 'Bordeaux', 'Toulouse', 'Nantes'];
const seedRides = async (count = 50) => {
const rides = Array.from({ length: count }, () => ({
driver_id: faker.string.uuid(),
origin: faker.helpers.arrayElement(CITIES),
destination: faker.helpers.arrayElement(CITIES),

departure_at:	faker.date.soon({ days: 30 }),
seats_available:	faker.number.int({ min: 1, max: 4 }),
price_per_seat:	faker.number.float({ min: 5, max: 50, precision: 0.5 }),
is_electric:	faker.datatype.boolean({ probability: 0.4 }),
status:	'active',
});	
const { error } = await supabase.from('rides').insert(rides);	
if (error) console.error('Erreur seeding rides:', error);	
else console.log(`\${count} trajets générés avec succès.`);	
};	
seedRides(50);	

5. Tests et Qualité du Code

5.1. Stratégie de tests

La qualité du code est garantie par une suite de tests automatisés réalisés avec Vitest, un framework de tests compatible avec Vite qui offre une API identique à Jest. Les tests se divisent en deux catégories :

- Tests unitaires : vérifient le comportement isolé d'une fonction ou d'un module
- Tests d'intégration : vérifient l'interaction entre plusieurs modules (routes API, base de données)

Type de test	Portée	Outil utilisé
Tests unitaires	Fonctions de validation, utilitaires, middlewares	Vitest
Tests d'intégration	Routes API avec base de données de test	Vitest + Supertest
Tests end-to-end	Flux complets (connexion → réservation)	Planifiés (prochaine étape)

5.2. Tests unitaires avec Vitest

Les fonctions de validation et les middlewares sont testés de manière unitaire. L'exemple ci-dessous teste le middleware d'authentification :

// BackEnd/tests/auth.middleware.test.js
import { describe, it, expect, vi } from 'vitest';
import jwt from 'jsonwebtoken';
import { authenticate } from '../src/middlewares/auth.middleware';
describe('Middleware authenticate', () => {
it('renvoie 401 si aucun token n\'est fourni', () => {
const req = { headers: {} };

<code>const res = { status: vi.fn().mockReturnThis(), json: vi.fn() };</code>
<code>const next = vi.fn();</code>
<code>authenticate(req, res, next);</code>
<code>expect(res.status).toHaveBeenCalledWith(401);</code>
<code>expect(res.json).toHaveBeenCalledWith({ error: 'Token manquant.' });</code>
<code>expect(next).not.toHaveBeenCalled();</code>
<code>});</code>
<code>it('appelle next() si le token est valide', () => {</code>
<code> const token = jwt.sign({ id: '1', role: 'user' }, 'test_secret', { expiresIn: '1h' });</code>
<code> process.env.JWT_SECRET = 'test_secret';</code>
<code> const req = { headers: { authorization: `Bearer \${token}` } };</code>
<code> const res = {};</code>
<code> const next = vi.fn();</code>
<code> authenticate(req, res, next);</code>
<code> expect(next).toHaveBeenCalled();</code>
<code> expect(req.user).toBeDefined();</code>
<code> expect(req.user.role).toBe('user');</code>
<code>});</code>
<code>it('renvoie 403 si le token est invalide', () => {</code>
<code> const req = { headers: { authorization: 'Bearer token_invalide' } };</code>
<code> const res = { status: vi.fn().mockReturnThis(), json: vi.fn() };</code>
<code> const next = vi.fn();</code>
<code> authenticate(req, res, next);</code>

<code>expect(res.status).toHaveBeenCalledWith(403);</code>
<code>});</code>
<code>});</code>

5.3. Résultats des tests

L'exécution des tests se fait via la commande `npm run test` depuis le dossier BackEnd. Les résultats obtenus démontrent la couverture des cas nominaux et des cas d'erreur :

<code>PASS tests/auth.middleware.test.js</code>
<code>Middleware authenticate</code>
<code>✓ renvoie 401 si aucun token n'est fourni (3ms)</code>
<code>✓ appelle next() si le token est valide (2ms)</code>
<code>✓ renvoie 403 si le token est invalide (1ms)</code>
<code>PASS tests/validation.test.js</code>
<code>validateRegister</code>
<code>✓ détecte un email invalide (2ms)</code>
<code>✓ détecte un mot de passe trop court (1ms)</code>
<code>✓ détecte des mots de passe non concordants (1ms)</code>
<code>✓ retourne un objet vide si tout est valide (1ms)</code>
<code>Test Suites: 2 passed, 2 total</code>
<code>Tests: 7 passed, 7 total</code>
<code>Duration: 0.892s</code>

6. Déploiement et Infrastructure

6.1. Containerisation avec Docker

Docker permet de créer des environnements de développement et de production reproductibles, indépendants du système d'exploitation de la machine hôte. Le projet utilise Docker Compose pour orchestrer deux services : le front-end et le back-end.

docker-compose.yml (à la racine du projet)
version: '3.8'
services:
backend:
build: ./BackEnd
container_name: ecoride-api
ports:
- '3001:3001'
environment:
- PORT=3001
- JWT_SECRET=\${JWT_SECRET}
- SUPABASE_URL=\${SUPABASE_URL}
- SUPABASE_KEY=\${SUPABASE_KEY}
- MONGO_URI=\${MONGO_URI}
restart: unless-stopped
frontend:
build: ./FrontEnd
container_name: ecoride-app
ports:
- '5173:80'
depends_on:
- backend
restart: unless-stopped

Le Dockerfile du back-end utilise une image Node.js alpine (légère) et un build multi-étapes pour réduire la taille de l'image finale :

BackEnd/Dockerfile
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
FROM node:20-alpine
WORKDIR /app
COPY --from=builder /app/node_modules ./node_modules
COPY . .
EXPOSE 3001
CMD ["node", "src/index.js"]

6.2. Déploiement du front-end sur Vercel

Vercel est une plateforme de déploiement spécialisée dans les applications front-end. L'intégration GitHub permet un déploiement automatique à chaque push sur la branche main. La configuration nécessite de définir la variable d'environnement VITE_API_URL pointant vers l'URL de l'API en production.

Avantages de Vercel pour ce projet :

- CDN mondial pour des temps de chargement rapides
- HTTPS automatique avec Let's Encrypt
- Preview deployments sur chaque Pull Request
- Zero configuration pour les applications Vite/React

6.3. Déploiement du back-end sur Render

Render est une plateforme cloud qui permet de déployer des services web via Docker. Le back-end est déployé comme un « Web Service » avec le Dockerfile du projet. Les variables d'environnement (JWT_SECRET, SUPABASE_URL, etc.) sont configurées directement dans l'interface de Render, sans jamais exposer de données sensibles dans le code source.

Plateforme	Service déployé	Coût	Déclencheur
Vercel	Front-end React	Gratuit (Hobby plan)	Push sur main
Render	API Node.js/Express via Docker	Gratuit (plan Free)	Push sur main
Supabase	Base de données PostgreSQL	Gratuit (Free tier)	Manuel
MongoDB Atlas	Base de données logs NoSQL	Gratuit (M0 Cluster)	Manuel

6.4. Gestion des variables d'environnement

Aucune donnée sensible n'est stockée dans le dépôt Git. Le fichier .env est listé dans .gitignore et ne doit jamais être commité. La structure du fichier .env est documentée dans le README du projet pour permettre à un contributeur de configurer son environnement :

.env - Structure des variables d'environnement
PORT=3001
JWT_SECRET=votre_clef_secrete_jwt_longue_et_aleatoire
SUPABASE_URL=https://xxxxxxxxxxxxx.supabase.co
SUPABASE_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
MONGO_URI=mongodb+srv://user:password@cluster0.xxxxx.mongodb.net/
FRONTEND_URL=http://localhost:5173

Sécurité : principe du moindre privilège

La clé Supabase utilisée côté back-end est la clé de service (service_role), qui contourne les politiques RLS (Row Level Security). Elle ne doit jamais être exposée côté client. Côté front-end, seule la clé anonyme (anon key) est utilisée, avec des politiques RLS restreignant les données accessibles.

Conclusion

Le projet EcoRide m'a permis de mettre en pratique l'ensemble des compétences du référentiel DWWM dans un contexte réaliste et complet. De la conception de la base de données à la mise en production, en passant par le développement d'une API sécurisée et d'une interface utilisateur moderne, chaque étape a été une occasion d'apprendre et de consolider mes acquis.

Parmi les réalisations techniques majeures, je retiens notamment :

- La mise en place d'une architecture full-stack découplée et scalable, avec un front-end React et une API Express indépendants
- L'implémentation d'un système d'authentification robuste par JWT, avec gestion des rôles et sécurisation de toutes les routes sensibles
- L'utilisation simultanée de deux bases de données (PostgreSQL et MongoDB) pour répondre à des besoins différents : intégrité relationnelle d'un côté, flexibilité et performance d'écriture de l'autre
- La containerisation de l'application avec Docker, garantissant la reproductibilité de l'environnement et facilitant le déploiement
- L'écriture de tests automatisés avec Vitest, adoptant une approche Test-Driven Development (TDD) pour les fonctions critiques

Ce projet a également mis en lumière des axes d'amélioration. Le système de notifications en temps réel (WebSockets), le paiement intégré et la version mobile de l'application sont des évolutions naturelles qui apporteraient une valeur ajoutée significative à EcoRide. Sur le plan de la sécurité, le remplacement du localStorage par des cookies HTTPOnly pour le stockage du JWT constitue une priorité.

Au-delà des compétences techniques, ce projet m'a appris à gérer l'ensemble du cycle de vie d'un produit numérique en autonomie : de la prise de besoin à la mise en production. Cette

expérience m'a conforté dans mon souhait de m'orienter vers un poste de développeur full-stack, où la polyvalence et la compréhension globale du système sont des atouts indéniables.

Annexes

Annexe A — Schéma complet de la base de données

Le schéma ci-dessous représente l'ensemble des tables de la base de données PostgreSQL (Supabase) et leurs relations.

users	rides	bookings	reviews
_____	_____	_____	_____
id (PK)	← driver_id	id (PK)	id (PK)
email	id (PK)	← ride_id	← booking_id
password	origin	— passenger_id	author_id
username	destination	seats_reserved	target_id
role	departure_at	status	rating
credits	seats_available	created_at	comment
created_at	price_per_seat		created_at
	is_electric		
	status		
vehicles			

id (PK)			
user_id (FK → users.id)			
brand			
model			
color			
energy_type			

Annexe B — Points de terminaison de l'API (Routes)

Méthode	Endpoint	Auth requise	Description
POST	/api/auth/register	Non	Inscription d'un nouvel utilisateur

POST	/api/auth/login	Non	Connexion et génération du JWT
GET	/api/rides	Non	Liste des trajets (avec filtres)
GET	/api/rides/:id	Non	Détails d'un trajet
POST	/api/rides	Oui (user)	Publier un nouveau trajet
POST	/api/rides/:id/book	Oui (user)	Réserver un trajet
GET	/api/users/me	Oui (user)	Profil de l'utilisateur connecté
PUT	/api/users/me	Oui (user)	Modifier son profil
GET	/api/admin/stats	Oui (admin)	Statistiques de la plateforme
GET	/api/admin/users	Oui (admin)	Liste de tous les utilisateurs
PATCH	/api/admin/users/:id/suspend	Oui (admin)	Suspendre un compte
GET	/api/admin/logs	Oui (admin)	Logs d'activité (MongoDB)

Annexe C — Commandes utiles

— Développement —————
npm run dev # Lance front + back simultanément
— Docker —————
docker compose up --build # Démarre l'application containerisée
docker compose down # Arrête et supprime les containers
— Tests —————
cd BackEnd && npm run test # Lance la suite de tests
npm run test:watch # Mode watch (rechargement automatique)
— Base de données —————

cd BackEnd && npm run seed:all	# Remplit toutes les tables
cd BackEnd && npm run db:clear	# Vide toutes les tables
cd BackEnd && npm run db:count	# Affiche le nombre de lignes

Annexe D — Technologies utilisées (récapitulatif)

Catégorie	Technologie	Version	Licence
Front-end	React	24.x	MIT
Front-end	Vite	5.x	MIT
Front-end	Tailwind CSS	3.x	MIT
Front-end	Framer Motion	11.x	MIT
Back-end	Node.js	20 LTS	MIT
Back-end	Express	4.x	MIT
Back-end	JSON Web Token	9.x	MIT
Back-end	bcrypt	5.x	MIT
Base de données	PostgreSQL (Supabase)	15.x	Open Source
Base de données	MongoDB Atlas	7.x	SSPL
Tests	Vitest	1.x	MIT
Infrastructure	Docker	25.x	Apache 2.0